

MODUL 3 INTERRUPT DAN TIMER

Abyan Devadi (H1H024049)

Asisten: Athallah Tsany Satriyaji

Tanggal Percobaan: Rabu, 13 Mei 2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Pemrograman – Jurusan Informatika UNSOED

Abstrak

Interrupt dan timer adalah dua mekanisme dasar dalam sistem embedded yang digunakan untuk merespons kejadian eksternal dan mengelola waktu tanpa menghambat eksekusi program utama. Praktikum ini mengimplementasikan keduanya pada Arduino Uno melalui dua percobaan: percobaan pertama menggunakan external interrupt dengan fungsi attachInterrupt() untuk mengendalikan LED melalui push button tanpa metode polling, dan percobaan kedua menggunakan fungsi millis() sebagai timer non-blocking agar LED berkedip secara periodik tanpa delay(). Interrupt terbukti membuat mikrokontroler merespons sinyal eksternal secara langsung, sementara pendekatan millis() menjaga program utama tetap berjalan di antara setiap pengecekan interval. Kedua mekanisme ini menjadi dasar yang perlu dipahami sebelum merancang sistem embedded yang lebih kompleks.

Kata Kunci: Interrupt, Timer, ISR, External Interrupt, millis(), Non-Blocking, Arduino, Embedded System, Polling

1. PENDAHULUAN

Sistem embedded kini hadir di hampir semua aspek kehidupan, dari perangkat IoT rumah tangga, sistem pemantauan industri, alat medis, sampai otomasi kontrol. Semakin banyak fungsi yang harus ditangani, semakin penting pula cara sistem mengelola kejadian dan waktu. Dua mekanisme paling dasar untuk kebutuhan ini adalah interrupt dan timer.

Pemrograman mikrokontroler konvensional biasanya mengandalkan metode polling, yaitu memeriksa kondisi input secara terus-menerus di dalam loop utama. Cara ini memang sederhana, tapi tidak efisien karena CPU terus bekerja meskipun kondisi yang diperiksa belum berubah. Semakin banyak sensor atau perangkat yang dipantau, semakin besar pula beban komputasi yang harus ditanggung.

Interrupt mengatasi keterbatasan polling dengan cara yang berbeda. Alih-alih terus-menerus memeriksa kondisi input, mikrokontroler cukup menjalankan program utamanya sampai sinyal interrupt datang. Saat itu terjadi, eksekusi berhenti sementara, ISR (Interrupt Service Routine) dijalankan, lalu program utama dilanjutkan dari

titik yang sama. Respons yang dihasilkan lebih cepat dan lebih deterministik dibanding polling, dengan penggunaan CPU yang jauh lebih efisien.

Timer adalah modul internal mikrokontroler yang menghitung waktu berdasarkan sinyal clock sistem. Di Arduino, pengelolaan waktu paling umum dilakukan dengan fungsi delay(), tapi fungsi ini bersifat blocking karena menghentikan seluruh eksekusi program selama durasi yang ditentukan. Fungsi millis() menawarkan pendekatan berbeda: program tetap berjalan sambil memeriksa apakah interval yang diinginkan sudah tercapai.

Praktikum ini mencakup dua percobaan pada Arduino Uno. Percobaan 6A mengendalikan kondisi LED menggunakan external interrupt yang dipicu push button, sehingga perubahan LED terjadi instan tanpa polling. Percobaan 6B menggunakan millis() sebagai timer non-blocking agar LED berkedip secara periodik tanpa delay(), dengan sistem yang tetap responsif selama timer berjalan.

2. STUDI PUSTAKA

2.1 KONSEP DASAR INTERRUPT DAN TIMER PADA MIKROKONTROLER

Interrupt adalah mekanisme dalam arsitektur mikrokontroler yang memungkinkan perangkat atau sinyal eksternal menginterupsi program utama untuk menangani kejadian tertentu. Ada dua jenis interrupt utama: Maskable Interrupt yang bisa dinonaktifkan lewat perangkat lunak, seperti INT0, INT1, dan Timer/Counter, serta Nonmaskable Interrupt yang tidak bisa dihalangi perangkat lunak, seperti sinyal reset [5]. Setiap sumber interrupt dapat diprogram ke dalam satu atau dua tingkat prioritas menggunakan register IP, sehingga interrupt berprioritas rendah bisa diinterupsi oleh yang berprioritas lebih tinggi, sedangkan yang tertinggi tidak bisa diinterupsi sama sekali [5].

Timer adalah modul internal mikrokontroler yang menghitung waktu berdasarkan sinyal clock sistem, secara independen dari eksekusi instruksi. Pada mikrokontroler AT89C51, Timer/Counter 16-bit

bekerja setiap 1 mikrodetik pada frekuensi 12 MHz, dan ketika periode tertentu terpenuhi, modul timer langsung menginterupsi mikrokontroler sebagai tanda bahwa perhitungan waktu selesai [5]. Kombinasi interrupt dan timer inilah yang membuat sistem kontrol berbasis waktu bisa merespons kondisi eksternal tanpa membuang siklus CPU untuk menunggu.

2.2 INTERRUPT SERVICE ROUTINE (ISR) DAN MEKANISME EXTERNAL INTERRUPT

Interrupt Service Routine (ISR) adalah fungsi khusus yang dieksekusi otomatis ketika sinyal interrupt datang. Program utama berhenti sementara, lalu melompat ke ISR yang sesuai. Setelah ISR selesai, program utama dilanjutkan dari titik yang sama, dengan instruksi Return From Interrupt (RETI) sebagai penanda akhir ISR [5]. Dengan cara ini, respons terhadap kejadian eksternal terjadi secara deterministik tanpa mengganggu alur program utama.

Dalam praktiknya, ISR harus dirancang sesingkat dan seefisien mungkin karena program utama sepenuhnya terhenti selama ISR berjalan. Penelitian pada sistem monitoring purging bag filter menunjukkan bahwa penggunaan fungsi `pulseIn()` yang bersifat blocking menyebabkan delay rata-rata hampir 1000 milidetik per penambahan sensor, sehingga dengan 8 sensor, total waktu pembacaan mencapai lebih dari 8 detik [1]. Ketika fungsi `pulseIn()` digantikan dengan mekanisme interrupt, waktu pembacaan program tetap konstan di sekitar 1,085 detik meski jumlah sensor bertambah, karena interrupt tidak memblokir eksekusi program dan langsung memberikan sinyal tanpa menunggu timeout [1]. Ini menunjukkan seberapa jauh perbedaan performa antara interrupt dan polling dalam hal kecepatan dan efisiensi CPU.

2.3 TIMER DAN METODE NON-BLOCKING PADA SISTEM EMBEDDED

Ada dua pendekatan umum untuk pengelolaan waktu pada sistem embedded: metode blocking dengan `delay()` dan metode non-blocking dengan timer berbasis `millis()` atau hardware timer. Fungsi `delay()` menghentikan seluruh eksekusi program selama durasi yang ditentukan, sehingga tidak cocok untuk sistem yang harus tetap responsif. Pendekatan non-blocking bekerja berbeda, yaitu program terus berjalan sambil memeriksa apakah selisih antara `millis()` saat ini dengan waktu yang dicatat sebelumnya sudah mencapai interval yang diinginkan.

Efektivitas timer non-blocking sudah diuji dalam sejumlah aplikasi nyata. Pada penelitian perancangan programmable timer berbasis ESP32-C3 Supermini, sistem menggunakan hardware timer dan pendekatan non-blocking untuk mengelola countdown dengan deviasi rata-rata hanya ± 1 detik per jam operasi [2]. Sistem tersebut mampu berjalan secara stabil selama 12 jam nonstop tanpa error, membuktikan keandalan pendekatan timer non-blocking untuk aplikasi kendali waktu yang presisi [2]. Penggunaan hardware timer juga menjadi prasyarat utama dalam membangun sistem penjadwalan yang akurat pada skala detik hingga menit tanpa memerlukan RTOS yang berat, selama latensi ISR terkontrol dan jitter diperhitungkan dalam perancangan perangkat lunak [2].

2.4 PENGARUH BLOCKING DELAY TERHADAP RESPONSIVITAS MIKROKONTROLER

Blocking delay adalah masalah nyata dalam perancangan sistem IoT industri berbasis mikrokontroler. Sumbernya bisa bermacam-macam, mulai dari penanganan interrupt yang tidak efisien, kontesi sumber daya, sampai kebijakan penjadwalan yang kurang optimal, dan semuanya berdampak langsung pada kemampuan sistem memenuhi deadline waktu nyata [4]. Tinjauan sistematis terhadap 50 penelitian di bidang ini menunjukkan bahwa hardware optimization seperti Direct Register Access (DRA) dan parallel context saving mampu mengurangi blocking delay hingga 60-75%, sementara algoritma penjadwalan adaptif berbasis interrupt nesting secara signifikan meningkatkan kepatuhan terhadap deadline eksekusi task [4].

Pada Arduino, dampak blocking delay terlihat jelas ketika membandingkan `pulseIn()` dengan mekanisme interrupt. Ketika delay dibiarkan terakumulasi dalam satu siklus pembacaan sensor, alat yang dibuat tidak akan beroperasi secara optimal karena setiap penambahan sensor meningkatkan total waktu tunggu secara linear [1]. Sebaliknya, mekanisme interrupt memungkinkan mikrokontroler untuk segera merespons sinyal tanpa menghambat proses lainnya, menjadikan sistem lebih responsif dan efisien [1]. Temuan ini selaras dengan literatur yang menyebutkan bahwa penanganan interrupt yang tepat merupakan cara paling efektif memitigasi blocking delay pada sistem IoT industri yang membutuhkan kepatuhan terhadap persyaratan real-time [4].

2.5 PENERAPAN INTERRUPT DAN TIMER PADA SISTEM IOT DAN OTOMASI

Interrupt dan timer digunakan secara luas dalam sistem IoT dan otomasi. Pada sistem smart home berbasis Arduino ESP8266 NodeMCU, fitur timer menjadwalkan kapan perangkat listrik menyala dan mati secara otomatis, sehingga konsumsi daya dapat dikurangi hingga 12,01% dalam satu bulan operasi [3]. Pengguna bisa mengonfigurasi jadwal on/off lewat antarmuka website, sementara timer bekerja otomatis tanpa interaksi manusia, menunjukkan bahwa mekanisme timer berbasis mikrokontroler bisa diterapkan untuk aplikasi efisiensi energi di lingkungan rumah tangga [3].

Pada skala industri, tuntutan presisi dan keandalan terhadap interrupt dan timer menjadi lebih tinggi. Sistem monitoring purging pada bag filter pabrik menggunakan fungsi interrupt pada NodeMCU ESP8266 untuk memantau 8 unit sensor secara bersamaan dan mengirimkan data ke platform IoT Thingier.IO setiap 0,5 detik, dengan tingkat keberhasilan penyimpanan data mencapai 96,6% [1]. Penerapan interrupt dalam sistem ini memungkinkan patroller mengetahui kondisi setiap unit purging secara real-time tanpa perlu melakukan pemeriksaan manual, yang sebelumnya membutuhkan waktu 30 menit hingga 1 jam dengan kemungkinan kesalahan 2 hingga 3 kali [1]. Selain itu, pada sistem kontrol berbasis mikrokontroler AT89C51, penggabungan modul Timer/Counter dengan interrupt handler memungkinkan komunikasi dua arah antara mikrokontroler dan perangkat seluler melalui layanan SMS, sehingga sistem dapat merespons perintah dari jarak jauh secara tepat waktu dan akurat [5].

3. METODOLOGI

3.1 ALAT DAN BAHAN

Praktikum ini terdiri dari dua percobaan dengan kebutuhan komponen yang sedikit berbeda. Untuk Percobaan 6A (External Interrupt), komponen yang digunakan meliputi satu unit Arduino Uno, satu buah push button sebagai sumber sinyal interrupt eksternal, satu buah LED sebagai indikator output, resistor 220 Ω untuk membatasi arus LED, resistor 10k Ω sebagai resistor pull-down pada rangkaian push button, kabel jumper, dan satu buah breadboard. Perangkat lunak yang digunakan adalah Arduino IDE beserta Serial Monitor bawaan untuk memantau output secara real-time. Untuk Percobaan 6B (Timer Menggunakan millis()), komponen yang digunakan adalah satu unit Arduino Uno, satu buah LED, resistor 220 Ω , kabel

jumper, dan satu buah breadboard. Perangkat lunak yang digunakan sama dengan Percobaan 6A: Arduino IDE dan Serial Monitor bawaan.

3.2 LANGKAH KERJA

Percobaan 6A mengendalikan kondisi LED menggunakan external interrupt yang dipicu push button. Setelah semua alat disiapkan dan Arduino Uno terhubung ke komputer via USB, program ditulis di Arduino IDE dengan menyertakan library Arduino.h sebagai header utama.

Variabel ledState bertipe bool dideklarasikan dengan keyword volatile agar compiler tahu bahwa nilainya bisa berubah kapan saja dari luar alur normal program, khususnya dari dalam ISR. ISR sendiri didefinisikan dengan nama tombolInterrupt() dan bertugas melakukan toggle pada ledState setiap kali interrupt dipanggil. Di dalam setup(), pin 13 dikonfigurasi sebagai OUTPUT dan pin 2 sebagai INPUT_PULLUP sehingga resistor internal Arduino aktif untuk pull-up. Fungsi attachInterrupt() dipanggil dengan parameter digitalPinToInterrupt(2) sebagai pin interrupt, tombolInterrupt sebagai nama ISR, dan FALLING sebagai mode pemicu, artinya interrupt aktif saat sinyal pada pin 2 berubah dari HIGH ke LOW ketika tombol ditekan. Di dalam loop(), nilai ledState ditulis ke pin 13 dengan digitalWrite() secara terus-menerus agar LED selalu mencerminkan kondisi variabel tersebut. Sketch disimpan dengan nama modul6_interrupt, lalu rangkaian dirakit di atas breadboard: LED ke pin 13, push button ke pin 2, serta GND dan 5V ke rel yang sesuai. Program dikompilasi, diunggah, kemudian tombol ditekan berulang kali untuk memastikan LED berpindah kondisi ON/OFF secara responsif setiap kali interrupt terpicu.

Percobaan 6B membuat LED berkedip secara periodik menggunakan millis() tanpa delay(). Setelah alat disiapkan dan Arduino Uno terhubung ke komputer, program mulai ditulis.

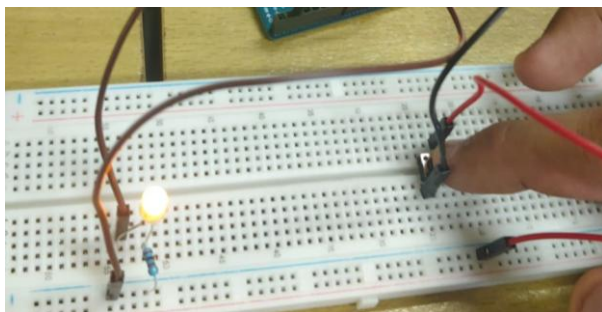
Tiga variabel dideklarasikan secara global: previousMillis bertipe unsigned long untuk menyimpan waktu terakhir LED berubah kondisi, interval bertipe const long bernilai 1000 sebagai periode kedip dalam milidetik, dan ledState bertipe bool untuk menyimpan kondisi LED. Di dalam setup(), pin 13 dikonfigurasi sebagai OUTPUT. Logika utama ada di fungsi loop(). Nilai millis() saat ini dibaca ke variabel currentMillis, lalu selisihnya dengan previousMillis dibandingkan terhadap interval. Jika selisih itu sudah mencapai atau melebihi 1000 milidetik, previousMillis diperbarui, ledState di-toggle, dan nilainya ditulis

ke pin 13 dengan `digitalWrite()`. LED pun berkedip setiap 1 detik tanpa satu pun pemanggilan `delay()`, sehingga `loop()` tidak pernah terblokir. Sketch disimpan dengan nama `modul6_timer`, lalu rangkaian dirakit: LED ke pin 13 dan GND ke rel ground breadboard. Program dikompilasi dan diunggah, kemudian diamati untuk memastikan LED berkedip secara konsisten setiap 1 detik sesuai nilai interval yang ditentukan.

4. HASIL DAN ANALISIS

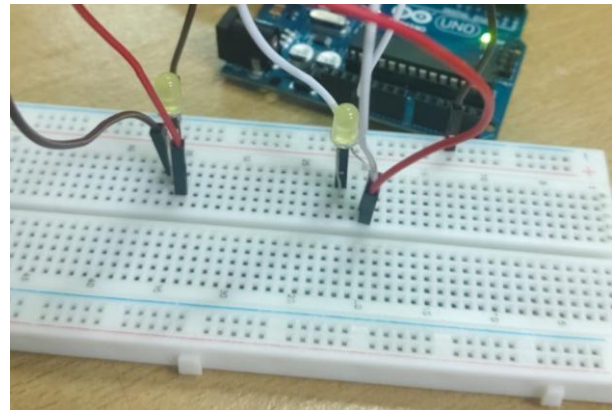
4.1 HASIL PERCOBAAN

Modul 6 dilaksanakan dalam dua percobaan. Pada Percobaan 6A, ISR bernama `tombolInterrupt()` didaftarkan melalui `attachInterrupt()` pada pin 2 dengan mode pemicuan `FALLING`, artinya ISR dieksekusi setiap kali tegangan pada pin 2 berpindah dari `HIGH` ke `LOW` saat tombol ditekan. Variabel `ledState` dideklarasikan sebagai `volatile bool` agar perubahannya di dalam ISR tetap terbaca oleh loop utama. Di dalam ISR, `ledState` di-toggle setiap penekanan, sementara `loop()` hanya bertugas menulis nilainya ke pin 13 dengan `digitalWrite()`. Setiap penekanan tombol langsung mengubah kondisi LED tanpa jeda yang bisa dirasakan, mengonfirmasi bahwa eksekusi ISR berjalan di luar alur `loop()` dengan prioritas lebih tinggi.



Gambar 1. Percobaan 6A

Pada Percobaan 6B, timer berbasis `millis()` diimplementasikan untuk mengedipkan LED di pin 13 dengan interval 1000 ms tanpa `delay()`. Variabel `previousMillis` menyimpan waktu terakhir LED berubah kondisi, sementara `currentMillis` diambil di setiap iterasi `loop()`. Selisih keduanya dibandingkan dengan nilai interval untuk menentukan kapan `ledState` di-toggle dan diterapkan ke pin 13. LED berkedip secara konsisten setiap satu detik, dan `loop()` tidak pernah terblokir sehingga bebas digunakan untuk tugas lain.



Gambar 2. Percobaan 6B

4.2 KEUNTUNGAN INTERRUPT DIBANDING POLLING

Interrupt dan polling adalah dua mekanisme yang berbeda untuk merespons kejadian eksternal. Pada polling, mikrokontroler secara aktif memeriksa status kondisi atau pin dalam setiap siklus `loop()`. CPU harus terus mengalokasikan waktu eksekusi untuk mengecek apakah suatu event terjadi, meski belum tentu muncul. Semakin banyak kondisi yang dipantau, semakin besar pula porsi CPU yang habis untuk pemeriksaan itu.

Interrupt bekerja sebaliknya. CPU cukup menjalankan program utama sampai sinyal interrupt datang dari hardware. Saat itu terjadi, CPU menyimpan konteksnya, menjalankan ISR, lalu melanjutkan dari titik yang sama. Tidak ada pemeriksaan yang sia-sia, latensi respons rendah dan deterministik, dan program utama bisa berjalan penuh di antara event. Percobaan 6A menunjukkan ini secara langsung: LED merespons setiap penekanan tombol secara instan karena `tombolInterrupt()` dipanggil hardware melalui mekanisme `FALLING` edge, bukan menunggu giliran di dalam `loop()`.

4.3 PENTINGNYA TIMER DALAM SISTEM EMBEDDED DAN REAL-TIME

Hampir semua operasi mikrokontroler bergantung pada ketepatan waktu, dan di sinilah timer berperan. Banyak tugas bersifat periodik: pembacaan sensor pada interval tertentu, pengiriman data berkala, kendali aktuator berbasis waktu, atau pembaruan tampilan. Tanpa timer yang andal, tugas-tugas ini tidak bisa berjalan tepat waktu.

Penggunaan `delay()` memang sederhana, tapi ia memblokir seluruh eksekusi `loop()` selama penundaan berlangsung. Tidak ada input yang bisa direspons, tidak ada sensor yang bisa dibaca, dan tidak ada tugas lain yang bisa dijalankan. Pada sistem real-time, ini menjadi masalah serius. Timer

berbasis `millis()`, seperti pada Percobaan 6B, mengatasi ini dengan menyimpan referensi waktu dan membandingkan selisihnya di setiap iterasi, sehingga `loop()` tidak pernah terblokir. Pada sistem yang lebih kompleks, pendekatan serupa diterapkan melalui timer hardware dan interrupt periodik agar eksekusi terjadi tepat waktu tanpa bergantung pada kecepatan `loop()`. Inilah mengapa timer menjadi dasar dari determinisme waktu pada sistem embedded.

4.4 ALUR KERJA SISTEM JIKA INTERRUPT DAN TIMER DIGABUNG

Ketika interrupt dan timer digabungkan, sistem bisa menangani dua jenis event sekaligus: yang asinkron dan tidak terprediksi, serta yang periodik dan terjadwal. Alur kerjanya terdiri dari beberapa lapisan eksekusi.

Pada lapisan paling dasar, `loop()` berjalan terus dan menangani logika utama program, termasuk memantau interval timer dan mengeksekusi tugas periodik seperti pembacaan sensor atau pembaruan tampilan. Di atas lapisan itu, mekanisme interrupt berjalan dengan prioritas lebih tinggi. Kapan saja event eksternal terjadi, baik penekanan tombol, sinyal sensor, maupun data serial yang masuk, hardware langsung memicu ISR yang menghentikan sementara semua yang sedang berjalan di `loop()`. ISR kemudian mengeksekusi respons secepat mungkin, biasanya sekadar mengubah flag atau variabel status, lalu menyerahkan kendali kembali ke titik semula. `Loop()` membaca flag itu dan menindaklanjutinya bersama logika timer yang melanjutkan tugasnya. Pola ini memungkinkan sistem tetap responsif terhadap event tak terduga tanpa mengorbankan ketepatan waktu operasi periodik.

4.5 DAMPAK ISR YANG TERLALU PANJANG ATAU KOMPLEKS

ISR harus dieksekusi secepat mungkin karena selama ISR berjalan, program utama dan interrupt berprioritas lebih rendah sepenuhnya tertunda. Jika ISR berisi logika panjang, seperti komputasi berat, operasi Serial, pengiriman data jaringan, atau bahkan `delay()`, beberapa masalah serius bisa timbul.

Yang pertama dan paling jelas adalah hilangnya responsivitas. `Loop()` terblokir selama ISR berjalan, sehingga semua tugas yang bergantung padanya, termasuk rutinitas timer berbasis `millis()`, ikut tertunda dan determinisme waktu yang ingin dicapai justru rusak. Selain itu, ada risiko kehilangan interrupt berikutnya: jika ISR terlalu lama dan interrupt baru dari sumber yang sama datang sebelum yang pertama selesai, interrupt itu

terlewat begitu saja. Yang lebih diam-diam berbahaya adalah ketidakstabilan nilai `millis()` itu sendiri. Fungsi tersebut tidak diperbarui selama ISR berjalan karena mekanisme internalnya bergantung pada interrupt timer yang ikut terblokir, sehingga nilai yang dibaca setelahnya bisa tidak konsisten.

Praktik terbaiknya sederhana: jaga ISR sesingkat mungkin, cukup ubah variabel volatile atau naikkan flag, lalu serahkan sisanya ke `loop()` yang membaca dan menindaklanjuti flag itu. Percobaan 6A menerapkan ini dengan tepat: ISR `tombolInterrupt()` hanya melakukan satu toggle pada `ledState`, dan `loop()` yang menangani aksi aktualnya lewat `digitalWrite()`.

5. KESIMPULAN

Praktikum modul 6 menunjukkan bahwa interrupt dan timer bekerja dengan cara yang berbeda tapi saling melengkapi. Percobaan 6A membuktikan bahwa `attachInterrupt()` mampu merespons penekanan tombol secara instan tanpa membebani `loop()`. ISR yang singkat dan hanya memodifikasi variabel volatile membuat sistem tetap stabil dan responsif. Percobaan 6B menunjukkan bahwa `millis()` lebih cocok untuk operasi periodik dibanding `delay()`, karena `loop()` tetap bisa berjalan di antara setiap pengecekan interval. Memahami cara kerja interrupt dan timer, termasuk mengapa ISR harus ringkas, adalah bekal dasar yang perlu dikuasai sebelum merancang sistem embedded yang lebih kompleks.

DAFTAR PUSTAKA

- [1] A. Sihab, S. Prasetya, dan R. Karimak, *Rancang Bangun System Monitoring Purging Pada Bag Filter Berbasis IoT (Internet of Things)*, Prosiding Seminar Nasional Teknik Mesin, Politeknik Negeri Jakarta, hal. 617–624, 2022.
- [2] M. D. Kurniawan dan A. Budjiyanto, *Implementasi Sistem Programable Timer Menggunakan ESP32-C3 Supermini dengan Tampilan OLED dan Indikator Buzzer*, JATI (Jurnal Mahasiswa Teknik Informatika), vol. 9, no. 6, hal. 9950–9956, Desember 2025.
- [3] D. A. Laksono, A. Widiyanto, dan B. Pujiarto, *Implementation of IoT to Reduce the Use of Electrical Energy*, Borobudur Informatics Review, vol. 01, no. 01, hal. 37–46, 2021.
- [4] P. Susanto, W. I. Kusumawati, Harianto, H. Pratikno, dan H. Prastyo, *Blocking Delay Effects on Microcontroller Speed and Responsiveness in Industrial IoT Devices: A Systematic Review*, Journal of Technology and Informatics (JoTI), vol. 8, no. 1, hal. 37–49, April 2026.

- [5] I. N. P. Sastra, D. M. Wiharta, dan A. Supranartha, *Perancangan dan Pembuatan Sistem Kontrol dengan Memanfaatkan Layanan SMS Telepon Selular Berbasis Mikrokontroler AT89C51*, Teknologi Elektro, vol. 4, no. 2, hal. 22–27, Juli–Desember 2005.